

```
{\rtf1\ansi\ansicpg1252\deff0\nouicompat\deflang1033{\fonttbl{\f0\fnil\fcharset0 Calibri;}}
{\*\generator Node.js RTF Converter}\viewkind4\uc1
\pard\sa200\sl276\slmult1\f0\fs24\par
\pard\qc\b\fs36 GuruBramha Platform - Database Documentation\b0\fs24\pard\sa200\sl276\slmult1\par
\par\b\fs32 1. Database Overview\b0\fs24\par
- \b Database Name\b0 : GuruBramha (Configurable via "MONGODB_URI")\par
- \b Purpose of the database\b0 : To securely manage and persist all platform data including user identities\b0\fs24\par
- \b Technologies used\b0 : MongoDB (NoSQL Database), Mongoose (Node.js ODM)\par
- \b Connection structure\b0 : The application connects to a MongoDB cluster using a secure connection string\b0\fs24\par
-----\par
\par\b\fs32 2. Complete Collections Structure\b0\fs24\par
\par\b\fs28 2.1 Users Collection (`users`) \b0\fs24\par
Stores user identities, academic profiles, external portfolio links, and subscription details.\par
- \b Fields & Data Types\b0 :\par
- "_id": ObjectId (Primary Key, Auto-generated)\par
- "firebaseUid": String (Required, Unique, Indexed)\par
- "displayName": String (Required)\par
- "email": String (Required, Unique, Indexed)\par
- "photoURL", "phone", "gender", "college", "branch", "academicYear", "district", "state", "postalCode": String (Optional)\par
- "age": Number (Optional)\par
- "githubUrl", "leetcodeUrl", "codechefUrl": String (Optional)\par
- "isSubscribed": Boolean (Default: "false")\par
- "subscriptionEndDate": Date (Optional)\par
- "scholarPoints": Number (Default: "0")\par
- "currentStreak": Number (Default: "0")\par
- "solvedProblems": Array of ObjectIds (Ref: "Problem")\par
- "bookmarkedCompanies": Array of ObjectIds (Ref: "Company")\par
- "createdAt", "updatedAt": Date (Auto-generated)\par
\par\b\fs28 2.2 Courses Collection (`courses`) \b0\fs24\par
Stores details for the elite course library and curriculum.\par
- \b Fields & Data Types\b0 :\par
- "_id": ObjectId (Primary Key)\par
- "title": String (Required)\par
- "instructor": String (Required)\par
- "category": String (Required)\par
- "level": String (Required, Enum: "Beginner", "Intermediate", "Advanced")\par
- "price": Number (Required)\par
- "originalPrice": Number (Optional)\par
- "duration": String (Optional)\par
- "rating", "reviewsCount", "studentsEnrolled": Number (Default: "0")\par
- "thumbnail", "description": String (Optional)\par
- "lessons": Array of Objects:\par
- "title" (String, Required)\par
- "duration", "videoUrl" (String, Optional)\par
- "isFreePreview" (Boolean, Default: "false")\par
\par\b\fs28 2.3 Problems Collection (`problems`) \b0\fs24\par
Stores coding challenges for the Elite Practice Arena.\par
- \b Fields & Data Types\b0 :\par
- "_id": ObjectId (Primary Key)\par
- "title": String (Required)\par
- "difficulty": String (Required, Enum: "Easy", "Medium", "Hard")\par
- "category": String (Required)\par
- "description": String (Required)\par
- "tags": Array of Strings\par
- "accuracy": Number (Default: "0")\par
- "points": Number (Default: "100")\par
- "validationCases": Array of Objects:\par
- "input", "output" (String)\par
- "isHidden" (Boolean, Default: "false")\par
\par\b\fs28 2.4 Companies Collection (`companies`) \b0\fs24\par
Stores interview preparation roadmaps and placement protocols.\par
- \b Fields & Data Types\b0 :\par
- "_id": ObjectId (Primary Key)\par
- "name": String (Required)\par
- "type": String (Required, Enum: "Product Based", "Service Based", "Startup")\par
- "difficulty": String (Enum: "Easy", "Medium", "Hard", "Expert")\par
- "logo", "package", "overview", "eligibility": String (Optional)\par
- "skills", "process": Array of Strings\par
- "rounds": Array of Objects ("title" String, "topics" Array of Strings)\par
- "roadmap30Days": Array of Objects ("week" Number, "focus" String)\par
\par\b\fs28 2.5 Transactions Collection (`transactions`) \b0\fs24\par
Stores Razorpay payment and subscription details.\par
- \b Fields & Data Types\b0 :\par
- "_id": ObjectId (Primary Key)\par
- "user": ObjectId (Required, Foreign Key -> "User")\par
- "amount": Number (Required)\par
```

```

- "currency": String (Default: "INR")\par
- "razorpayOrderId": String (Required, Indexed)\par
- "razorpayPaymentId", "razorpaySignature", "receipt": String (Optional)\par
- "status": String (Enum: "created", "successful", "failed", Default: "created")\par
\par\b\fs28 2.6 Documents Collection (`documents`) \b0\fs24\par
Stores metadata for certificates fetched via DigiLocker.\par
- \b Fields & Data Types\b0 :\par
- "_id": ObjectId (Primary Key)\par
- "user": ObjectId (Required, Foreign Key -> "User")\par
- "digilockerId": String (Required)\par
- "name", "uri": String (Required)\par
- "type": String (Optional)\par
- "issueDate": Date (Optional)\par
\par -----\par
\par\b\fs32 3. User Side Database Structure\b0\fs24\par
From the user's perspective, the database structure facilitates the following:\par
- \b Authentication/Login\b0 : Managed via Firebase on the frontend, which syncs with the "users" collection
- \b Profile Data\b0 : Users can update their academic history, age, gender, and social links (GitHub, Leet
- \b Courses\b0 : Users browse the "courses" collection. Premium courses check the "user.isSubscribed" flag
- \b Subscriptions & Payments\b0 : Initiating a purchase creates a "transaction" document. Upon success via
- \b Practice/Coding\b0 : Users query the "problems" collection. Successful submissions append the Problem'
- \b Documents\b0 : Users authenticating with DigiLocker create records in the "documents" collection, link
\par -----\par
\par\b\fs32 4. Admin Side Database Structure\b0\fs24\par
The admin perspective provides full control over the platform's content and tracking:\par
- \b User Management\b0 : Admins can query the "users" collection to monitor "scholarPoints", view academic
- \b Content Management\b0 : Admins have CRUD access to "courses", "problems", and "companies". They can ad
- \b Transactions & Reports\b0 : Admins query the "transactions" collection filtered by "status: 'successfu
- \b Analytics\b0 : Aggregating "user.solvedProblems", "course.studentsEnrolled", and "problem.accuracy" he
\par -----\par
\par\b\fs32 5. Flow Charts\b0\fs24\par
""mermaid\par
graph TD\par
    %% User Registration / Login Flow\par
    subgraph Registration & Login\par
        A1[User opens App] --> A2{Has Account?}\par
        A2 -- No --> A3[Firestore Signup]\par
        A3 --> A4[Backend creates Document in 'Users' Collection]\par
        A2 -- Yes --> A5[Firestore Login]\par
        A5 --> A6[Backend fetches 'Users' Document]\par
    end\par
    %% Course Purchase & Subscription Flow\par
    subgraph Payment & Subscription\par
        B1[User clicks Buy Subscription] --> B2[Backend creates Razorpay Order]\par
        B2 --> B3[Create pending record in 'Transactions' Collection]\par
        B3 --> B4[User completes Payment via Razorpay UI]\par
        B4 --> B5[Backend verifies Signature /verify route]\par
        B5 --> B6[Update Transaction status to 'successful']\par
        B6 --> B7[Update User 'isSubscribed' to true]\par
    end\par
    %% Document Upload / DigiLocker Flow\par
    subgraph DigiLocker Flow\par
        C1[User clicks Link DigiLocker] --> C2[Redirect to DigiLocker OAuth]\par
        C2 --> C3[User Approves Access]\par
        C3 --> C4[Backend receives Callback Code]\par
        C4 --> C5[Backend exchanges Code for Access Token]\par
        C5 --> C6[Fetch Issued Documents]\par
        C6 --> C7[Save metadata to 'Documents' Collection]\par
    end\par
""\par
\par -----\par
\par\b\fs32 6. API & Backend Mapping\b0\fs24\par
The frontend communicates with the MongoDB collections via the following expected REST API mapping:\par
- \b Authentication / Profile\b0 \par
- "POST /api/auth/sync" -> Maps to "User.findOneOrCreate()" \par
- "GET /api/users/profile" -> Maps to "User.findById()" \par
- "PUT /api/users/profile" -> Maps to "User.findByIdAndUpdate()" \par
- \b Courses\b0 \par
- "GET /api/courses" -> Maps to "Course.find()" \par
- "GET /api/courses/:id" -> Maps to "Course.findById()" (Protected payload checks "User.isSubscribed") \par
- \b Practice Arena\b0 \par
- "GET /api/problems" -> Maps to "Problem.find()" \par
- "POST /api/problems/:id/submit" -> Verifies code, updates "User.solvedProblems" & "User.scholarPoints" \par
- \b Interview Roadmaps\b0 \par
- "GET /api/companies" -> Maps to "Company.find()" \par
- "POST /api/companies/:id/bookmark" -> Updates "User.bookmarkedCompanies" \par

```

```

- \\b Payments ("/api/payment")\\b0 \\par
- "POST /order" -> Uses Razorpay API, creates "Transaction" doc\\par
- "POST /verify" -> Verifies signature, updates "Transaction" status, updates "User" subscription\\par
- \\b DigiLocker ("/api/digilocker")\\b0 \\par
- "GET /authorize" -> Redirects to Gov portal\\par
- "GET /callback" -> Processes OAuth token\\par
- "GET /documents" -> Fetches external API, saves to "Documents"\\par
\\par -----\\par
\\par\\b\\fs32 7. Deployment Readiness\\b0\\fs24\\par
\\par\\b\\fs28 Environment Variables Required (`.env` file)\\b0\\fs24\\par
""`env\\par
PORT=5000\\par
MONGODB_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/gurubramha?retryWrites=true&w=majority\\pa
FRONTEND_URL=https://your-frontend-domain.com\\par
\\pard\\qc\\b\\fs36 Razorpay Keys\\b0\\fs24\\pard\\sa200\\sl276\\slmult1\\par
RAZORPAY_KEY_ID=your_key_id\\par
RAZORPAY_KEY_SECRET=your_key_secret\\par
\\pard\\qc\\b\\fs36 DigiLocker Keys\\b0\\fs24\\pard\\sa200\\sl276\\slmult1\\par
DIGILOCKER_CLIENT_ID=your_client_id\\par
DIGILOCKER_CLIENT_SECRET=your_client_secret\\par
DIGILOCKER_REDIRECT_URI=http://localhost:5000/api/digilocker/callback\\par
\\pard\\qc\\b\\fs36 Firebase Admin SDK\\b0\\fs24\\pard\\sa200\\sl276\\slmult1\\par
FIREBASE_PROJECT_ID=your_project_id\\par
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\\n...\\n-----END PRIVATE KEY-----\\n"\\par
FIREBASE_CLIENT_EMAIL=your_client_email\\par
""`\\par
\\par\\b\\fs28 Database Setup Instructions\\b0\\fs24\\par
1. Create a MongoDB Atlas cluster or install MongoDB locally.\\par
2. Obtain the connection string and insert it into "MONGODB_URI".\\par
3. In "index.js", the database connects automatically.\\par
4. (Optional) Create a seed script using Mongoose to insert initial courses and problems into the collections
\\par\\b\\fs28 Production-Ready Checks & Security\\b0\\fs24\\par
- \\b Network Security\\b0 : Ensure MongoDB Atlas Network Access is IP-restricted (e.g., only Azure server IP
- \\b CORS\\b0 : The "cors" configuration in "index.js" correctly restricts origins to trusted frontend domain
- \\b Environment Secrets\\b0 : Ensure ".env" is listed in ".gitignore" and keys are managed securely in Azure
- \\b Indexes\\b0 : Mongoose schemas have implicit indexes (e.g., "unique: true" on emails), ensuring fast queries
- \\b Validation\\b0 : Mongoose enforces schema validation (Required fields, Enums, Number formats) preventing
}

```